Parallel Programming in the Multi-Hole Mie Pipeline

How TB-scale top-view spray videos are reduced to plume-wise 1D metrics

Linan Jiang

Aalto University

May 2026

The Runtime Result We Care About

Current production-style timing

$$731.91\ \mathrm{s} + 963.31\ \mathrm{s} = 1695.22\ \mathrm{s} \approx 28.25\ \mathrm{min}.$$

This is for the TB-scale multi-hole Mie video archive after switching off expensive optional outputs.

Option	Current
`repair_bw`	False
`save_boundary_points_csv`	False
`save_preview_avi`	False
`preview_playback`	False

Meaning: the timing mostly reflects the core computation, not preview videos or boundary-point file dumping.

One-Sentence Summary

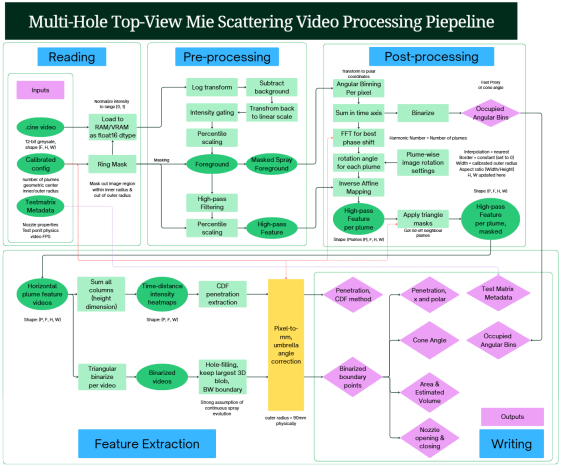
The pipeline is not just a faster loop.

It is an assembly line:

- GPU: does the dense pixel/frame math.
- CPU threads: compute per-plume engineering metrics.
- Background I/O: reads the next video and writes the current results.

The main goal is simple: keep the expensive machine busy and avoid waiting for disk or optional visualization.

The Actual Processing Pipeline



The image-processing chain converts one normalized `.cine` video into plume-wise time series and exported CSV/metadata.

Why a MATLAB-Style Loop Is Not Enough

Natural MATLAB view

1. loop over videos;
2. loop over frames;
3. loop over pixels;
4. loop over plumes;
5. save results.

Problem at archive scale

- The same operation is repeated for millions of pixels.
- Disk reading and CSV writing interrupt computation.
- Plume-wise features are independent but slow if serialized.
- Optional QC output can dominate runtime if left on.

The fix is to parallelize by data shape, not by adding more nested loops.

Layer 1: One Backend, Two Machines

Backend abstraction

The code uses a single array namespace, `xp`. If CUDA/CuPy is available, `xp` means GPU arrays; otherwise it falls back to NumPy.

Concept	Practical meaning
`np.asarray(...)`	CPU memory, normal NumPy/MATLAB-like array.
`cp.asarray(...)`	GPU memory, same array idea but stored on the graphics card.
`xp.log`, `xp.sum`	Write one operation; run it on the active backend.
`__cuda_array_interface__`	Quick test: is this array already on the GPU?

Code anchors: `OSCC_postprocessing/utils/backend.py`, `mie_multi_hole.py:_cpu_to_backend`.

Layer 2: GPU Tensor Work

What stays on the GPU

Once a video is loaded, the heavy image operations are kept in one memory space as large tensors.

- normalize 12-bit camera data;
- log-domain background subtraction;
- median background estimation;
- noise-floor gating;
- Sobel high-pass filtering;
- ring/chamber masking.
- angular density accumulation;
- triangle thresholding;
- largest-component cleanup;
- CDF-front penetration;
- plume strip stack: (P, F, H, W);
- robust percentile scaling.

For a mechanical analogy: the GPU is doing the same machining operation on many workpieces at once.

Layer 3: Multi-Hole Geometry Makes Parallelism Useful

Physical structure

A multi-hole injector naturally gives repeated plume directions around a common nozzle center.

- transform full-frame video into nozzle-centered angular density;
- estimate rotational offset from the plume-count harmonic;
- rotate/crop each plume into a local strip;
- stack all strips into a 4D tensor.

Why this matters

After registration, every plume has the same local coordinate system:

$$\text{plume tensor} = (P,\ F,\ H,\ W).$$

Then reductions such as sum-over-height, CDF-front location, and per-plume feature extraction become regular operations.

Code anchor: `OSCC_postprocessing/analysis/mie_multihole.py:mie_multihole_postprocessing`.

Layer 4: CPU Threads for Plume-Wise Metrics

Not every step should stay on the GPU

After binarization, some engineering metrics are easier and more stable as CPU-side plume-wise feature extraction.

- area;
- BW penetration;
- polar penetration;
- cone-angle proxies;
- opening/closing timing;
- optional boundary points.

Code anchor: `mie_multi_hole.py:_collect_metric_columns`.

Threading choice

The code converts each plume mask to host memory, then runs plume feature extraction with a `ThreadPoolExecutor`.

This avoids Windows process-spawn overhead and avoids duplicating large plume videos across worker processes.

Layer 5: Hide Disk I/O Behind Computation

Read-ahead

While video N is being processed on the GPU, a single background worker reads video $N + 1$ from disk.

- depth-1 prefetch;
- skipped files are not prefetched;
- host-to-GPU copy stays on the main thread.

Write-behind

Metrics CSV and metadata JSON are submitted to a writer pool, so the next GPU job can begin without waiting for disk writes.

- 3-worker I/O writer pool;
- pending writes drained at folder end;
- optional preview and boundary output are gated by switches.

Code anchors: `_get_prefetch_executor`, `_get_io_writer_executor`, `_process_parent_folder`.

What the Disabled Options Save

Option	What it does	Why it costs time
`repair_bw`	3D morphology and largest connected component cleanup	extra binary image operations over every plume/frame.
`save_boundary_points_csv`	saves detailed boundary point clouds	many extra coordinates and extra files/NPZ writes.
`save_preview_avi`	writes tiled QC videos	video encoding is slow and disk-heavy.
`preview_playback`	opens interactive OpenCV playback	blocks the main pipeline; no overlap.

Presentation wording

For production throughput, I disabled optional diagnostic outputs and measured the core path: load, preprocess, register, segment, reduce to metrics, and export CSV/metadata.

Where the Time Goes in One Video

The code logs each video with the same timing breakdown:

Timing field	Meaning
`load`	decode `.cine`, normalize, and move data to the active backend.
`preproc`	log subtraction, noise gating, Sobel high-pass, masks.
`postproc+cdf`	angular registration, plume strips, thresholding, CDF front.
`bw_metrics`	plume-wise binary-envelope features.
`save_csv_submit`	hand off CSV/metadata writing to background I/O.

This timing design is useful because it shows whether the bottleneck is disk, GPU image processing, or CPU feature extraction.

Engineering Mental Model

Simple loop

1. read one video;
2. process it;
3. save everything;
4. then start the next video.

This pipeline

1. read the next video in the background;
2. process the current video on GPU;
3. compute plume metrics in CPU threads;
4. save results in background threads;
5. reuse GPU memory within the folder.

The computation is organized like a production line, not like a single workbench.

Main Takeaways

1. **Dense image math belongs on the GPU.** Mie videos are naturally large arrays, and most preprocessing steps apply the same operation to every pixel/frame.
2. **Geometry makes the data regular.** Multi-hole registration converts a messy top-view image into plume-aligned tensors.
3. **Independent plume metrics can run in CPU threads.** Each plume is a separate strip after registration.
4. **I/O is overlapped rather than waited on.** Prefetch and writer pools keep disk access from idling the GPU.
5. **Optional outputs are separated from production throughput.** Preview videos and boundary point dumps are useful for QC, but they are not part of the fastest archive pass.

Code Map for Discussion

File/function	Role
`mie_multi_hole.py`	batch orchestration, config, prefetch, writer pool, checkpoints.
`_process_cine_file`	one-video end-to-end path.
`_cpu_to_backend`	CPU-to-GPU transfer, float16 conversion, normalization.
`mie_multihole_preprocessing`	log subtraction, Sobel, masks.
`mie_multihole_postprocessing`	angular density, FFT offset, plume strip extraction.
`penetration_cdf_all_plumes`	CDF-front penetration for all plumes.
`_collect_metric_columns`	CPU-threaded plume-wise BW metrics.
`_get_prefetch_executor` / `_get_io_writer_executor`	read-ahead and write-behind.

Suggested verbal close: "The result is a reproducible engineering data pipeline: it turns the image archive into plume-wise time series quickly enough that target definitions and model stages can be iterated."